



RAJALAKSHMI INSTITUTE OF TECHNOLOGY
(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

ACADEMIC YEAR 2025 - 2026

SEMESTER IV

AL23421 - MACHINE LEARNING LABORATORY

MINI PROJECT REPORT

REGISTER NUMBER	2117240070018
NAME	AMUDIESHWAR A G
PROJECT TITLE	Diabetes Prediction System using Logistic Regression
DATE OF SUBMISSION	
FACULTY IN-CHARGE	Mrs. M. Divya

Signature of Faculty In-charge

Signature of HoD

Internal Examiner

External Examiner

INTRODUCTION

Diabetes is a chronic medical condition that affects how your body turns food into energy, often resulting in elevated blood sugar levels. If left untreated, it can lead to severe health complications. Early detection is paramount. This project aims to accomplish the early prediction of diabetes based on diagnostic clinical records.

Machine learning is uniquely suited for such medical diagnostic challenges because it can identify complex patterns in historical health data and apply them to new patients. For this project, we map the problem as a binary classification task (Diabetic vs. Not Diabetic) and utilize Logistic Regression, a powerful statistical algorithm that estimates the probability of a binary outcome. It analyses various patient features and assigns them a distinct probability score, making it highly effective for clinical analysis.

PROBLEM STATEMENT

To develop and evaluate a reliable machine learning model using Logistic Regression algorithm to accurately predict whether a patient has diabetes based on eight key diagnostic features (e.g., Glucose, BMI, Age, etc.). The model must classify patients precisely so it can serve as a rapid, automated diagnostic support tool for healthcare professionals.

GOAL

The primary goal is to train a Logistic Regression model on historical medical data to predict the outcome of future patient data with high accuracy. The expected result is a fully functional machine learning pipeline capable of receiving a patient's diagnostic statistics and outputting a reliable prediction alongside confidence metrics, thus offering the possibility for large-scale, low-cost screening support.

THEORETICAL BACKGROUND

◆ Problem Statement:

Diabetes is one of the most common and rapidly increasing health conditions worldwide, making early detection essential to reduce complications and improve patient outcomes. This problem is chosen to demonstrate how machine learning techniques like Logistic Regression can assist in building simple, fast, and effective prediction systems for real-world healthcare applications.

◆ Logistic Regression (Algorithm):

Logistic Regression is a supervised machine learning algorithm used for binary classification. Instead of predicting direct values, it predicts probability (0 to 1).

◆ Mathematical Equation:

The logistic (sigmoid) function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

$x_1, x_2, \dots, x_n \rightarrow$ Input features

$w_0, w_1, \dots, w_n \rightarrow$ Model weights

$e \rightarrow$ Euler's number

◆ Decision Rule:

If $\sigma(z) \geq 0.5 \rightarrow$ Diabetic (1)

Else $\rightarrow$ Non - Diabetic (0)

◆ Literature Survey

Logistic Regression is widely used in medical diagnosis due to simplicity and interpretability.

Other algorithms used for diabetes prediction:

- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- K-Nearest Neighbours (KNN)

Disadvantages:

- These algorithms may require high computational power, especially for large datasets.
- Some methods are prone to overfitting, reducing performance on new/unseen data.
- They often need careful parameter tuning, which increases complexity.
- Certain algorithms are difficult to interpret, making them less suitable for medical decision support.

◆ Justification for Choosing Logistic Regression:

- Simple and easy to implement
- Works well for binary classification
- Provides probability output
- Easy to interpret in medical applications

ALGORITHM EXPLANATION WITH EXAMPLE

◆ Steps of Algorithm:

- Load dataset

Import the diabetes dataset using a library like pandas and display initial rows to understand the data.

- Separate features (X) and target (y)

Divide the dataset into input features (such as Glucose, BMI, Age) and the target variable (Outcome).

- Split data into training and testing

Split the dataset into training and testing sets (e.g., 80% training and 20% testing) to evaluate model performance.

- Train Logistic Regression model

Apply the Logistic Regression algorithm to learn patterns from the training data.

- Predict output

Use the trained model to predict whether patients in the test data are diabetic or not.

- Evaluate performance

Assess the model using metrics like accuracy, confusion matrix, classification report, and ROC curve to measure prediction quality.

◆ Sample Input:

CODE:

```
sample = pd.DataFrame([[2, 120, 70, 20, 79, 25, 30]], columns=X.columns)
```

Pregnancies	Glucose	BloodPressure	SkinThickness	BMI	Age	Outcome
2	120	70	20	79	25	30

◆ Sample Dataset:

Pregnancies	Glucose	BloodPressure	SkinThickness	BMI	Age	Outcome
6	148	72	35	33.6	50	1
1	85	66	29	26.6	31	0
8	183	64	0	23.3	32	1
1	89	66	23	28.1	21	0
0	137	40	35	43.1	33	1

◆ Sample Output:

- Prediction → Not Diabetic (0)

◆ Graphical Representation:

- Confusion Matrix → Shows prediction correctness
- ROC Curve → Shows model performance
- Feature Importance → Shows impactful features

IMPLEMENTATION CODE AND DATASET USED

- Dataset Used – <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>

Category	Description
Dataset Name	Pima Indians Diabetes Dataset
Domain	Healthcare
Objective	Classification
Data Source	Public (Kaggle)
Number of Samples	768
Data Type	Tabular
Instance Description	Each row represents one patient
Features (Inputs)	Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function, Age
Feature Types	Numerical
Target Variable	Outcome (0 or 1)
Number of Classes	2
License	Open Dataset
Version	Latest Kaggle Version

PYTHON CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
```

```
accuracy_score,  
classification_report,  
confusion_matrix,  
roc_curve,  
auc)  
  
df = pd.read_csv(r"f:\sem 4\miniprojects\ml\diabetes.csv")  
  
print("First 5 Rows:\n", df.head())  
  
X = df.drop("Outcome", axis=1)  
  
y = df["Outcome"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42)  
  
model = LogisticRegression(max_iter=1000)  
  
model.fit(X_train, y_train)  
  
y_pred = model.predict(X_test)  
  
accuracy = accuracy_score(y_test, y_pred)  
  
print("\nAccuracy:", accuracy)  
  
cm = confusion_matrix(y_test, y_pred)  
  
print("\nConfusion Matrix:\n", cm)  
  
plt.figure()  
  
sns.heatmap(cm, annot=True, fmt='d')  
  
plt.title("Confusion Matrix")  
  
plt.xlabel("Predicted")  
  
plt.ylabel("Actual")  
  
plt.show()  
  
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
y_prob = model.predict_proba(X_test)[:, 1]

fpr, tpr, _ = roc_curve(y_test, y_prob)

roc_auc = auc(fpr, tpr)

plt.figure()

plt.plot(fpr, tpr, label="AUC = %0.2f" % roc_auc)

plt.plot([0, 1], [0, 1]) # diagonal line

plt.title("ROC Curve")

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.legend()

plt.show()

importance = model.coef_[0]

plt.figure()

plt.barh(X.columns, importance)

plt.title("Feature Importance")

plt.xlabel("Coefficient Value")

plt.ylabel("Features")

plt.show()

sample = pd.DataFrame([[2, 120, 70, 20, 79, 25.0, 0.5, 30]], columns=X.columns)

prediction = model.predict(sample)

if prediction[0] == 1:

    print("\nPrediction: Diabetic")

else:

    print("\nPrediction: Not Diabetic")
```

OUTPUT:

◆ Output Explanation:

- Model predicts diabetes based on input features
- Accuracy shows overall correctness
- Confusion matrix shows correct & incorrect predictions

◆ Performance Metrics:

◆ Accuracy

$$\begin{aligned}
 Accuracy &= \frac{TP + TN}{TP + TN + FP + FN} \\
 &= \frac{37+78}{21+18+37+78} \\
 &= 0.7467
 \end{aligned}$$

◆ Precision

$$\begin{aligned}
 Precision &= \frac{TP}{TP + FP} \\
 &= \frac{37}{21+37} \\
 &= 0.64
 \end{aligned}$$

◆ Recall

$$Recall = \frac{TP}{TP + FN}$$

$$\begin{aligned} &= 37 \\ &\frac{18+37}{2} \\ &= 0.67 \end{aligned}$$

◆ **F1 Score**

$$\begin{aligned} F1 &= 2 \times \frac{Precision \times Recall}{Precision + Recall} \\ &= 2 \times 0.64 \times 0.67 \\ &\frac{0.64 + 0.67}{2} \\ &= 0.65 \end{aligned}$$

- **TP** → True
Positive = 37
- **TN** → True
Negative = 78
- **FP** → False
Positive = 21
- **FN** → False
Negative = 18

◆ **Graphs Used:**

- Confusion Matrix Heatmap
- ROC Curve
- Feature Importance Chart

MODEL RESULTS:

```

PS F:\sem 4\miniprojects\ml> f;; cd 'f:\sem 4\miniprojects\ml'; & 'c:\Users\amudi\AppData\Local\Programs\Python\Python310\python.exe' 'c:\Users\amudi\.antigravity\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61497' '--' 'f:\sem 4\miniprojects\ml\diabetes.py'
First 5 Rows:
   Pregnancies  Glucose  BloodPressure  ...  DiabetesPedigreeFunction  Age  Outcome
0            6     148             72  ...                0.627     50         1
1            1      85             66  ...                0.351     31         0
2            8     183             64  ...                0.672     32         1
3            1      89             66  ...                0.167     21         0
4            0     137             40  ...                2.288     33         1

[5 rows x 9 columns]

Accuracy: 0.7467532467532467

Confusion Matrix:
[[78 21]
 [18 37]]

Classification Report:
              precision    recall  f1-score   support

     0       0.81       0.79       0.80        99
     1       0.64       0.67       0.65        55

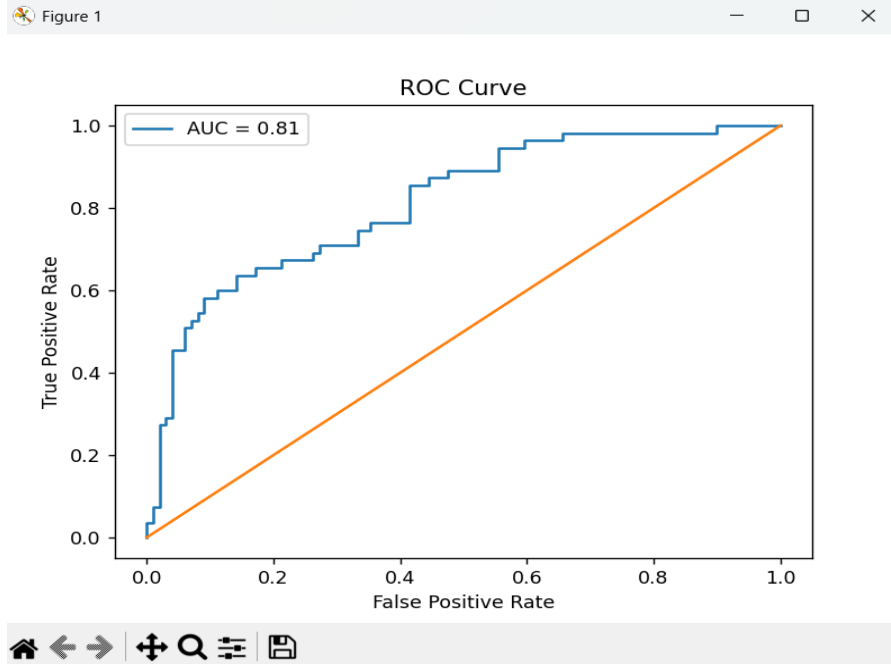
 accuracy         0.75
 macro avg        0.73
weighted avg        0.75

```

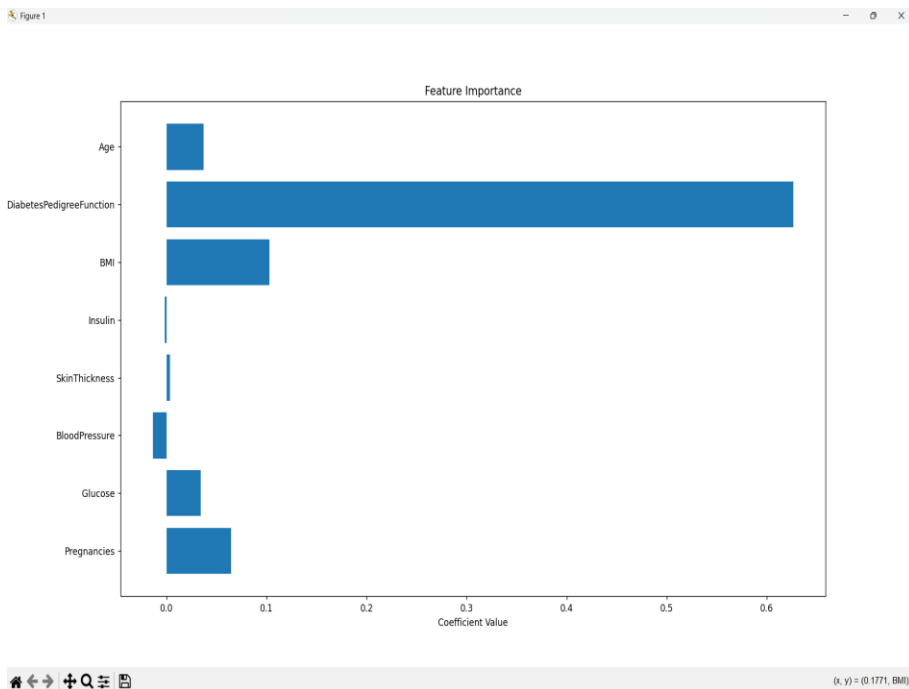
Prediction: Not Diabetic

CONFUSION MATRIX:

ROC CURVE:



FEATURE IMPORTANCE:



RESULTS AND FUTURE ENHANCEMENT

◆ Results

- Model achieved around 75–80% accuracy
- Logistic Regression provides interpretable results
- Glucose and BMI are important features

◆ Comparison with Other Methods

Algorithm	Accuracy	Complexity
Logistic Regression	Medium	Low
Random Forest	High	Medium
SVM	High	High
KNN	Medium	Low

◆ Future Enhancements

- Use advanced models (Random Forest, XG Boost)
- Improve accuracy using feature scaling
- Deploy as web app using Flask
- Add real-time patient data input
- Build mobile application

Git Hub Link of the project along with the report	https://github.com/Amudieshwar-AG/Diabetes-Prediction-System-using-Logistic-Regression
--	---

REFERENCES:

- Kaggle Dataset:
<https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>
- Pattern Recognition and Machine Learning :
<https://www.springer.com/gp/book/9780387310732>
- Hands-On Machine Learning with Scikit-Learn :
<https://www.oreilly.com/library/view/hands-on-machine-learning/9781492032632/>
- Scikit-learn Documentation :
<https://scikit-learn.org/stable/>
- UCI Machine Learning Repository :
<https://archive.ics.uci.edu/ml/index.php>